UNIVERSITY OF MAKATI
J.P. Rizal Extension, West Rembo, Makati City

College of Computing and Information Sciences

FINAL PROJECT IN MODELING AND SIMULATION

FIX IT WITH SCRUM: A GAMIFIED, WEB-BASED SANDBOX FOR LEARNING SCRUM AS THE SCRUM MASTER OF AN AI TEAM

NANAT, JAMES ADRIAN A.
CASINTO, DOMINIC G.
LIGAYA, DIANNE CHRISTEL
ORIAS, ETHAN MARCO
URMENITA, MORPHEUS EARL
III-CCSAD

Project Title: Simulating the Transactional Core of Fix it with Scrum

1. Introduction

1.1 System Description

Fix it with Scrum is a web-based, gamified learning sandbox for students of software engineering and algorithms. Behind a secure login, a student plays the Scrum Master of an AI-driven Scrum team (an AI Product Owner and two AI Developers) during a one-to-two-day mini-sprint on a small algorithms project, a delivery route planner. The student does not write code or assign work: the student facilitates the four sprint stages (Planning, Daily Standup, Build, and Review and Retrospective) and defends complexity decisions to the AI team. Performance is scored on two tracks, the team's algorithmic output and the student's Scrum process behaviour.

The platform is a decoupled web system. A browser client talks to an API gateway and orchestrator that keeps each agent's own conversation history, schedules scenario events and logs transcripts. A PostgreSQL database stores accounts, scores and transcripts. An external LLM provider API generates every agent turn. A stateless algorithm-engine service (`POST /api/v1/algorithm/solve`) runs Engine E1 (stage-constrained context window chaining) and Engine E2 (normalized learning gain and saturating process score). Code written by the AI Developers is autograded inside an isolated, ephemeral execution sandbox.

The purpose of the application is to give students on-demand practice at facilitating Scrum ceremonies and handling difficult team behaviour while probing algorithmic complexity, and to give instructors transcripts and process scores. The system is first evaluated in a pilot of 8–10 students and, if successful, may be offered to whole course sections.

1.2 Objectives

The objective of this simulation is to model the platform's transactional core (login, repeated AI-agent turns, code submission and autograding, final scoring) to find the maximum number of student sessions per hour the platform can serve before more than 5% of students abandon because an agent reply or an autograding result takes too long. The simulation answers these questions before the system is used beyond the pilot:

- What is the maximum load (students starting a session per hour) before the abandonment rate exceeds 5%?
- Which resource is the primary bottleneck: authentication workers, LLM provider call slots or sandbox workers? Are Engines E1 and E2 a constraint?

- How do turn response time, session time, throughput, utilization, queue length and abandonment change as load grows?
- Which fix is cheaper at launch-day load: adding resources (Option A) or applying E1 budgeted context to shorten every LLM call (Option B)?
- What capacity should be planned for the pilot and for a full course section?

1.3 Scope

The simulation models one student session from login to final score, and the competition between many simultaneous sessions for shared resources.

Included processes:

- Authentication, including failed logins and retries (Argon2id password verification)
- Agent turn: the student's reply, optional E1 context building, and the LLM provider call
- Repeated agent turns until the conversation ends (30 or 34 turns, depending on the student's persona)
- Code submission and sandbox autograding, including failed tests, revision and resubmission (at most three attempts)
- Final scoring by Engine E2
- Abandonment when an agent reply or an autograding result takes too long

Agent types: students (one entity per session, with a persona). The AI agents are represented by the LLM calls a session generates. **System components (resources):** authentication workers, LLM provider call slots, sandbox workers. **Time frame / scenarios:** baseline operations (24 h), stress test, optimization and scaling.

Excluded from scope:

- UI/UX design and game content (personas' dialogue, scenario-event scripts, pedagogy)
- Quality of the LLM replies, persona fidelity and learning gain (studied in the pilot)
- Correctness of Engines E1 and E2; only their measured run times are used
- Network latency, failures and restarts of any component; the optional Redis queue path; database contention (millisecond writes)
- Security attacks; rate limiting is represented only as capacity
- Instructor dashboards, exports and the pre/post assessment

2. Conceptual Model

2.1 Identification of the Transactional Core

The transactional core comprises the state-changing operations that every student session performs. Names in parentheses are the blocks of the AnyLogic flowchart.

- **Authentication and session start.** The student submits credentials; the gateway verifies the Argon2id hash and creates a server-side session. A failed login is retried after a short wait. (studentSource, seizeAuth, loginAuth, releaseAuth, checkLogin, loginRetry)
- **Agent turn (the repeated core loop).** The student reads the team's message and replies; with E1 enabled the orchestrator builds a token-budgeted context (Engine E1); the LLM provider generates the team's reply, which is logged. (studentThink, buildContext, seizeLLM, llmCall, releaseLLM, moreTurns)
- **Stage advance and process scoring.** The sprint stage moves along PLAN → STANDUP → BUILD → REVIEW (the E1 state machine) and each rated process event updates the E2 saturating accumulator. Both cost under a millisecond, so they are folded into the turn.
- **Code submission and autograding.** The team's `solution.py` runs in the sandbox against hidden tests and timing checks. If the tests fail, the team revises and resubmits, up to three attempts. (seizeSandbox, runTests, releaseSandbox, checkTests, reviseSolution)
- **Final scoring.** Engine E2 computes the output and process score. (computeScore, studentExit)
- **Abandonment.** A student who waits too long for an LLM slot or a sandbox worker leaves. (sinkAbandonedLLM, sinkAbandonedSbx)

2.2 Model Representation

The model is a discrete-event simulation of one process flow. Each student is an entity that moves through the steps below and competes with other students for three limited resources. Figures 1–3 show the activity flowcharts of the three transactions; Figure 4 shows their AnyLogic implementation.

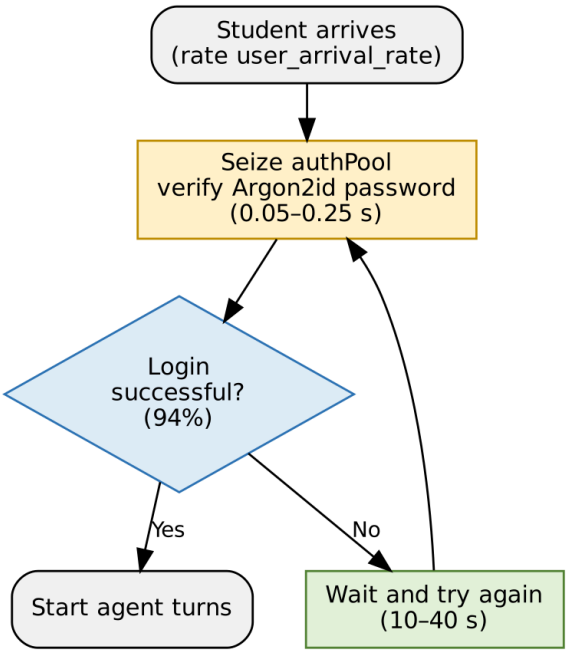


Figure 1. Login transaction (activity flowchart).

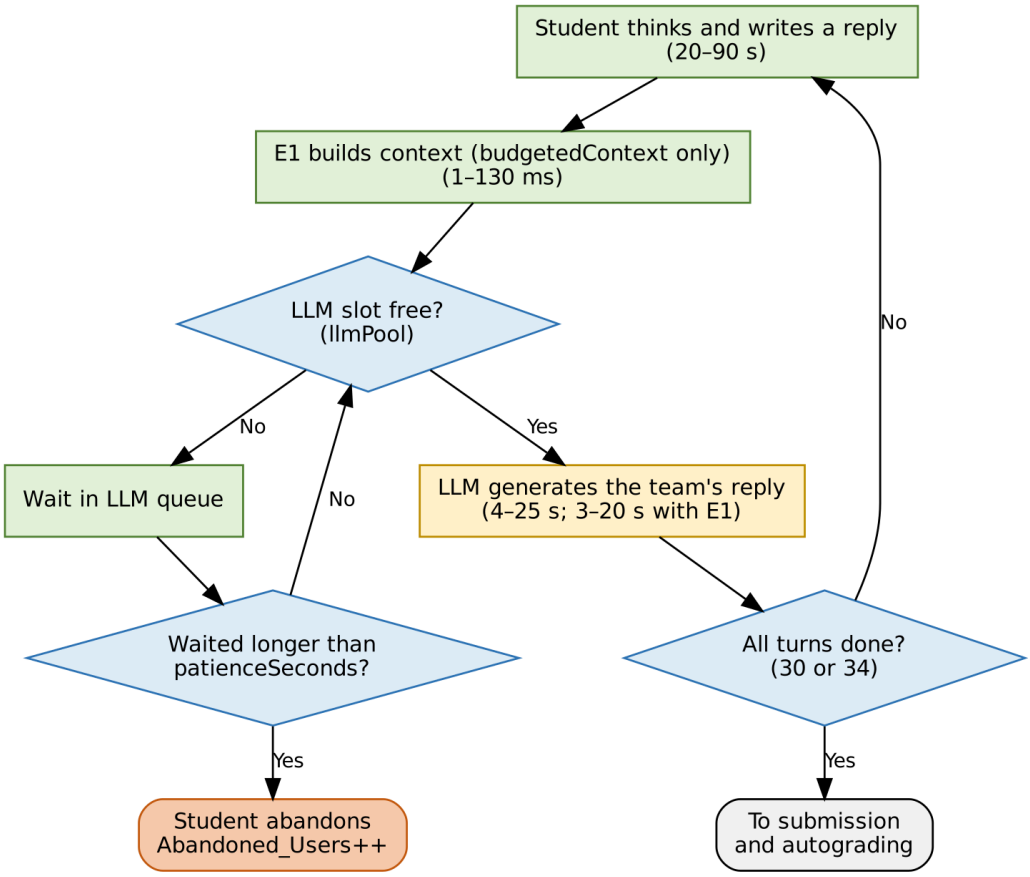


Figure 2. Agent turn transaction (activity flowchart).

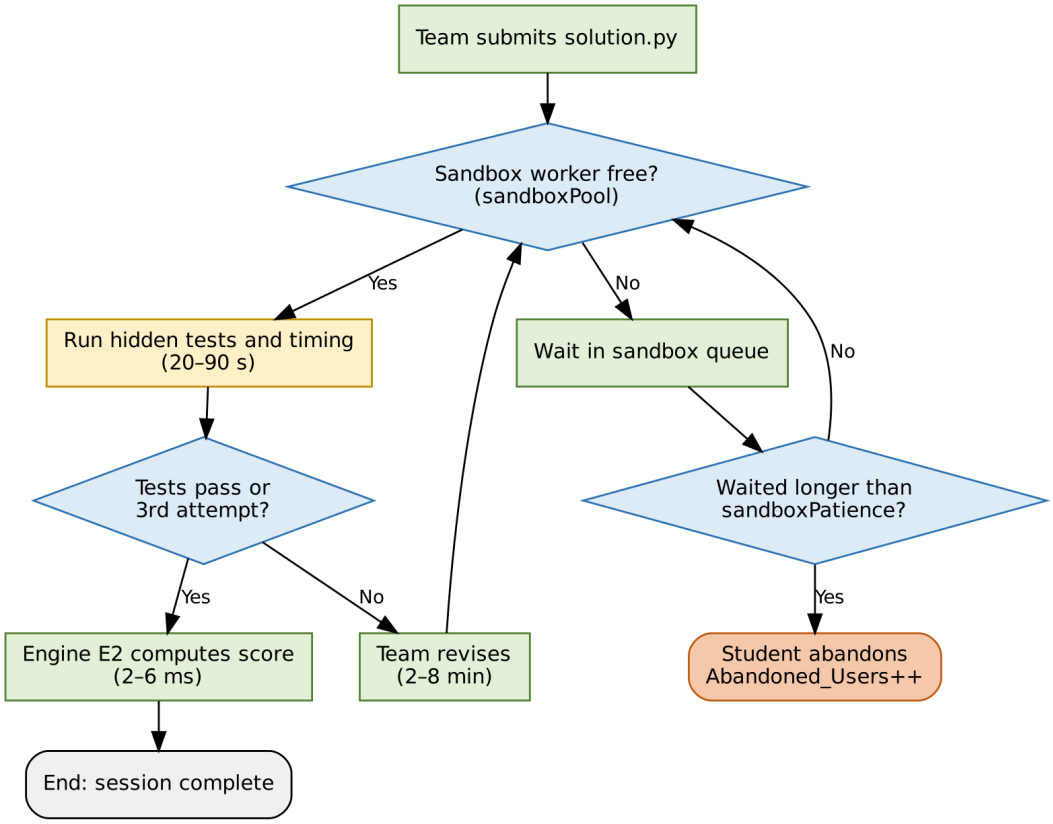


Figure 3. Submission and autograding transaction (activity flowchart).

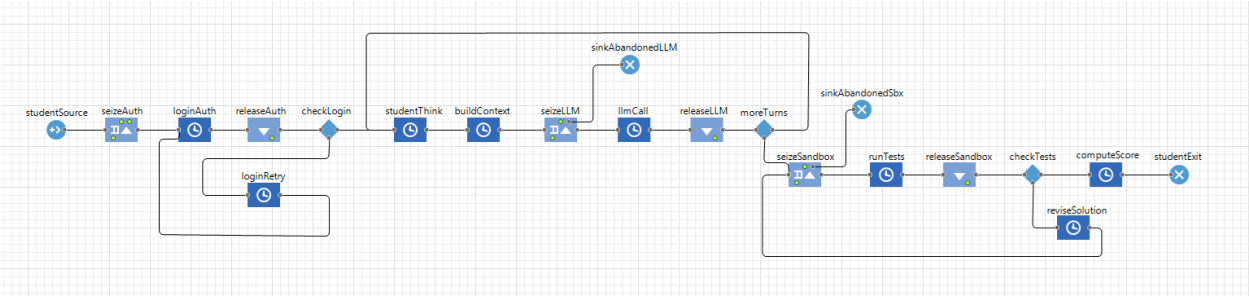


Figure 4. AnyLogic implementation of the three flowcharts.

Process flow

1. **studentSource:** students start a session at `user_arrival_rate` per hour (exponential inter-arrival times). Each student is a “lazy” (50%) or “perfectionist” (50%) persona, which sets the number of turns (34 or 30) and the probability that the tests pass (0.70 or 0.80).
2. **seizeAuth / loginAuth / releaseAuth:** the student takes an authentication worker (authPool) for password verification; triangular(0.05, 0.10, 0.25) s.
3. **checkLogin / loginRetry:** the login succeeds with probability 0.94; otherwise the student waits triangular(10, 20, 40) s and logs in again.

4. **studentThink:** the student reads the team's message and replies; triangular(20, 45, 90) s; no resource.
5. **buildContext:** if `budgetedContext` is true, Engine E1 selects the transcript segments for the prompt; triangular(0.001, 0.05, 0.13) s; otherwise 0.
6. **seizeLLM / llmCall / releaseLLM:** the student needs one LLM slot (llmPool); the call takes triangular(4, 10, 25) s, or triangular(3, 8, 20) s with E1 budgeted context. A student who waits longer than `patienceSeconds` (120 s) for a free slot leaves through sinkAbandonedLLM.
7. **moreTurns:** if `turnsDone` is below `turnsNeeded`, the student returns to studentThink.
8. **seizeSandbox / runTests / releaseSandbox:** the team's solution waits for a sandbox worker (sandboxPool) and runs hidden tests; triangular(20, 40, 90) s. A student who waits longer than `sandboxPatience` (300 s) leaves through sinkAbandonedSbx.
9. **checkTests / reviseSolution:** tests pass with the persona's probability, or the third attempt is accepted; otherwise the team revises for triangular(2, 4, 8) min and resubmits.
10. **computeScore / studentExit:** Engine E2 computes the score; triangular(0.002, 0.003, 0.006) s; the session time is recorded.

Resources (baseline capacities)

Resource pool	Represents Figure 2)	Baseline capacity
authPool	API gateway authentication workers (Argon2id verification)	2 (`authWorkers`)
llmPool	Concurrent calls allowed by the LLM provider (API rate limiter)	5 (`llmSlots`)
sandboxPool	Ephemeral, isolated code-execution containers	1 (`numSandboxWorkers`)

The algorithm engine (E1, E2) is not a pool: its calls take milliseconds against seconds for the LLM, so it is modelled as short delays and checked analytically in Section 5.2.

2.3 Assumptions

Technical assumptions

- LLM reply time is triangular (4, 10, 25) s (mean 13 s) and triangular (3, 8, 20) s (mean 10.3 s, about 20% shorter) with E1 budgeted context. The shorter call reflects that cost and latency grow with every input token
- The provider's rate limit is a fixed number of concurrent call slots; one student turn is one LLM call (the team's reply is generated in one call).

- Engine times come from the benchmarks: E1 about 1–130 ms and the E2 bootstrap for a pilot-size cohort (N = 10, L = 1,000) about 3 ms. Other service times are assumptions (Section 3.2).
- Network latency and database writes are negligible; no component fails or restarts during a run; resource capacities are fixed during a run.

User behaviour assumptions

- Students start sessions as a Poisson process with a constant rate over the 24 h run (no daily cycle); personas are split 50/50 and behave as described in Section 2.2.
- A session of about 32 turns is the part of the mini-sprint in which the student is actively chatting; per-sprint token totals (200,000–400,000, §3.8) are not modelled.
- A student who waits more than 120 s for an LLM slot, or 300 s for a sandbox worker, abandons the whole session and does not retry (a conservative simplification). Time spent waiting is not “served”, so abandonment frees capacity.
- If the three AI agents each made a separate LLM call per student turn, the LLM load would be about three times higher; this sensitivity case is discussed in Section 5.2.

3. Methodology

3.1 Modeling Approach

The project uses **discrete-event simulation (DES)**. The platform is a network of queues in which individual students move through sequential steps, wait for limited resources and sometimes give up. DES represents each student as an entity, captures queueing and resource contention at the authentication workers, the LLM provider and the sandbox, and measures waiting times, utilization and abandonment directly. Time unit: seconds; the clock advances from event to event (arrivals, service completions, timeouts).

Alternatives were not used because they hide the behaviour of interest: system dynamics aggregates individual students into flows and cannot represent abandonment of individuals; a pure agent-based model adds behavioural detail the platform does not need; and analytical queueing (Erlang formulas) needs assumptions that do not hold for closed-loop sessions with timeouts and resubmissions. Simple analytical laws (the utilization law and Little’s law) are used to validate the simulation (Section 5.3).

Stochastic elements: exponential inter-arrival times, triangular service and think times, persona, login success and test outcome. **Deterministic elements:** resource capacities, turn counts per persona, patience limits and the process logic.

3.2 Model Components

Entities. *Student*: one session. Attributes: `persona`, `turnsNeeded`, `turnsDone`, `passProb`, `attempts`, `testsPass`, `tStart` (session start), `tSent` (start of the current turn) and `tQ` (time it joined a queue).

Inputs and outputs. Inputs are the arrival rate, resource capacities, the E1 switch (`budgetedContext`), persona and patience parameters. Outputs are the metrics M1–M11 of Section 4.3.

Parameters and how they were determined:

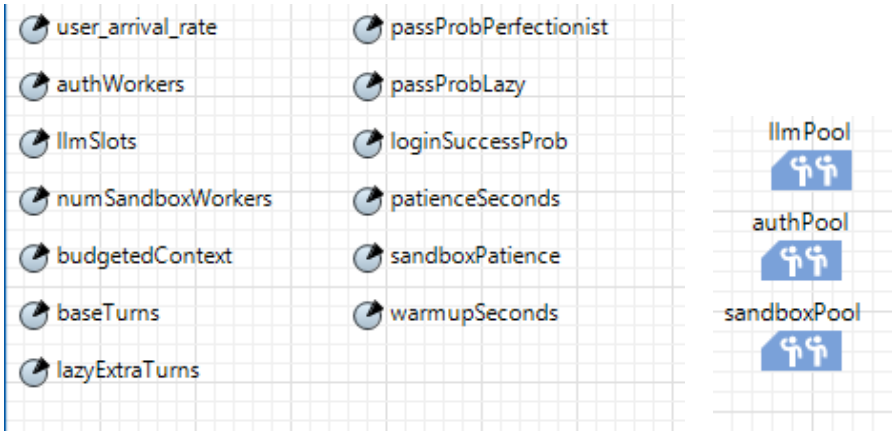


Figure 5. Model parameters and resource pools.

Parameter (AnyLogic name)	Baseline	Basis
`user_arrival_rate`	10 students/h	Assumption: a pilot-scale class (8–10 students) starting within the hour. Varied 5–80 in Experiment 2.
`authWorkers`, `llmSlots`, `numSandboxWorkers`	2, 5, 1	Assumed baseline capacities; varied in Experiment 3 (llmSlots 5–12, sandbox 1–4).
`budgetedContext`	false	false = no E1 (full-history prompts); true = E1 budgeted context (Option B).
`baseTurns`, `lazyExtraTurns`	30, 4	Assumption: about 30 conversation turns; the “lazy” persona needs 4 more.
`passProbLazy`, `passProbPerfectionist`	0.70, 0.80	Assumption.
`loginSuccessProb`	0.94	Assumption: 6% of logins fail (typos, expired sessions).
`patienceSeconds`, `sandboxPatience`	120 s, 300 s	Assumption: waiting time after which a student gives up.
`warmupSeconds`	1,800 s	Statistics are reset after 30 minutes (Section 4.4).

Service-time distributions:

Step	Resource	Triangular (min, mode, max)	Mean	Source
loginAuth	authPool	(0.05, 0.10, 0.25) s	0.13 s	Assumption: Argon2id at OWASP minimum settings takes tens of milliseconds.
loginRetry	none	(10, 20, 40) s	23.3 s	Assumption.
studentThink	none	(20, 45, 90) s	51.7 s	Assumption: reading and typing a reply.
buildContext (E1)	none	(0.001, 0.05, 0.13) s	0.06 s	**Measured** greedy about 1 ms; 48.7 ms at n = 200, B = 4,000; at most 130 ms at the policy threshold.
llmCall	llmPool	(4, 10, 25) s; with E1 (3, 8, 20) s	13.0 s; 10.3 s	Assumption (replace with pilot logs).
runTests	sandboxPool	(20, 40, 90) s	50.0 s	Assumption: hidden tests plus timing at growing input sizes (proposal §3.5).
reviseSolution	none	(2, 4, 8) min	4.7 min	Assumption.
computeScore (E2)	none	(0.002, 0.003, 0.006) s	0.004 s	**Measured**: E2 bootstrap for N = 10, L = 1,000 takes 2.88–2.91 ms

Data collection. The system is still a prototype, so no production logs exist. The process stages, turn counts and token budgets come from the testing we did, and the E1 and E2 run times from the benchmark scripts (`benchmark.py`, one development machine). Every other time is a planning estimate, marked above, and should be replaced by pilot measurements. The model records counts of arrivals, completions, abandonments, login retries and resubmissions; waiting and response times; utilization and queue lengths sampled every minute; and shows them on a live dashboard.

3.3 Tools and Technology

The simulation was built and run in **AnyLogic** (version 8.9.10) using the Process Modeling Library (Source, Seize, Delay, Release, SelectOutput, Sink, ResourcePool) and short Java actions. A Simulation experiment (10 runs) was used for Experiment 1, a Parameters Variation experiment (16 arrival rates × 10 replications = 160 runs) for Experiment 2, and an Optimization experiment (OptQuest demo engine, 32 combinations × 5 replications = 160 runs per option) for Experiment 3. The model was built in stages: a flow without resources, then the three resource pools, then abandonment and statistics, then the dashboard, then the experiments. Every run printed one result line to the console; the lines were copied to Excel, where the replications were averaged and the graphs drawn.

4. Simulation Model

4.1 Model Implementation

The model was implemented in AnyLogic with the Process Modeling Library and Java actions in the block properties. The **Main** agent holds the parameters, variables, statistics, resource pools, flowchart, dashboard and a reporting event. The **Student** agent type holds the per-student attributes of Section 3.2.

Block	Type	Logic
studentSource	Source	Rate `user_arrival_rate` per hour; assigns persona, `turnsNeeded`, `passProb`, `tStart`.
seizeAuth / loginAuth / releaseAuth	Seize, Delay, Release	authPool; triangular(0.05, 0.10, 0.25) s.
checkLogin / loginRetry	SelectOutput, Delay	Success with probability `loginSuccessProb`; otherwise wait triangular(10, 20, 40) s and log in again.
studentThink	Delay	triangular(20, 45, 90) s; records the send time.
buildContext	Delay	`budgetedContext` ? triangular(0.001, 0.05, 0.13) : 0 (seconds).
seizeLLM / llmCall / releaseLLM	Seize, Delay, Release	llmPool; `budgetedContext` ? triangular(3, 8, 20) : triangular(4, 10, 25) s. Seize has an exit on timeout after `patienceSeconds`.
sinkAbandonedLLM	Sink	Receives the timeout port of seizeLLM; counts an abandoned user.
moreTurns	SelectOutput	`turnsDone < turnsNeeded` loops back to studentThink.
seizeSandbox / runTests / releaseSandbox	Seize, Delay, Release	sandboxPool; triangular(20, 40, 90) s; timeout after `sandboxPatience`; attempts incremented.
sinkAbandonedSbx	Sink	Receives the timeout port of seizeSandbox.
checkTests / reviseSolution	SelectOutput, Delay	Pass (`testsPass`, drawn once in runTests) with `passProb` or on the 3rd attempt; otherwise revise triangular(2, 4, 8) min and resubmit.
computeScore / studentExit	Delay, Sink	E2 triangular(0.002, 0.003, 0.006) s; session time recorded at exit.

Key logic (Java actions). Counters and statistics are reset once, at the end of the warm-up period; a cyclic event samples utilization and queue lengths every 60 s; a reporting event prints one result line per run, which is copied to Excel.

```
// studentSource - On exit
nArrived++; agent.persona = randomTrue(0.5) ? 1 : 2; agent.tStart = time();
agent.turnsNeeded = baseTurns + (agent.persona == 1 ? lazyExtraTurns : 0);
agent.passProb = agent.persona == 1 ? passProbLazy : passProbPerfectionist;
// studentThink - On exit
agent.tSent = time();
// seizeLLM - On enter / On exit
nWaitLLM++; agent.tQ = time();   /   nWaitLLM--; statLLMWait.add(time() - agent.tQ);
// releaseLLM - On exit
agent.turnsDone++; nTurns++; statTurn.add(time() - agent.tSent);
// sinkAbandonedLLM - On enter   (sinkAbandonedSbx is analogous)
```

```
nWaitLLM--; nAbandonLLM++;
// runTests - On exit   (checkTests condition is simply agent.testsPass)
agent.testsPass = randomTrue(agent.passProb) || agent.attempts >= 3;
// KPI functions
int Abandoned_Users() { return nAbandonLLM + nAbandonSbx; }
double abandonPct() { int t = nCompleted + Abandoned_Users(); return t > 0 ? 100.0 *
Abandoned_Users() / t : 0; }
```

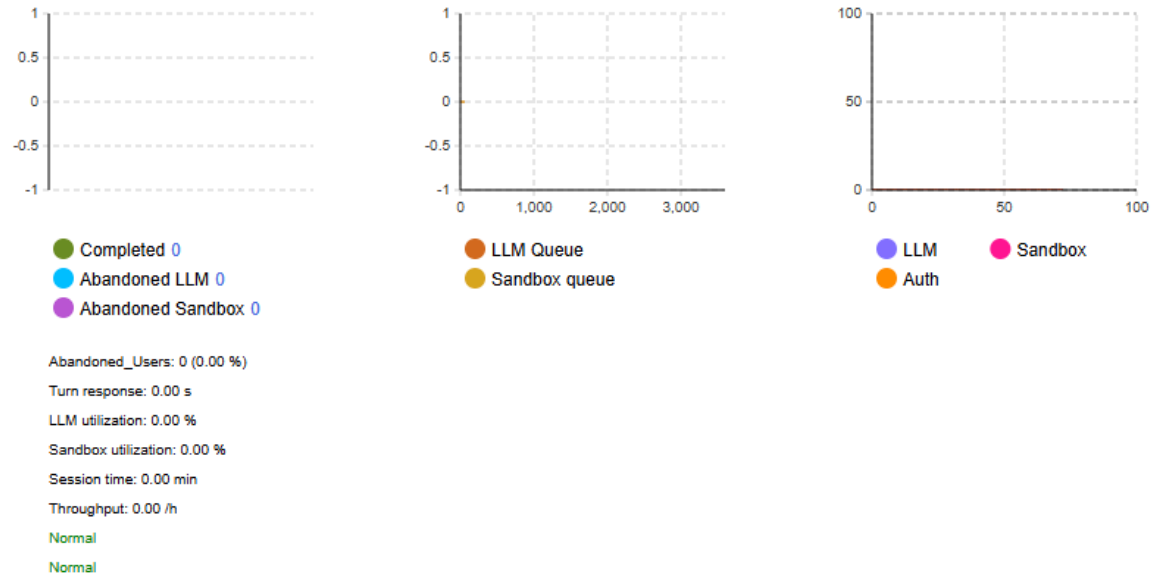


Figure 6. Dashboard.

Verification. The logic was checked against hand calculations and consistency checks on the recorded results (Section 5.3): the session time and turn response time at low load equal the sum of the mean service times; the measured login-retry and resubmission rates match the input probabilities; LLM and sandbox utilization at baseline agree with the utilization law; throughput equals the arrival rate when nothing is lost; and the saturation point agrees with the analytic estimate.

4.2 Scenarios

Three experiments were created and run on the same model. The independent variables are the arrival rate, the resource capacities and the E1 switch (`budgetedContext`).

Experiment 1: Baseline Analysis

Question: How does the system perform under the expected, normal user load?

Setup: Simulation experiment, 10 runs of 24 h; `user_arrival_rate` = 10 students/h; `llmSlots` = 5; `numSandboxWorkers` = 1; `authWorkers` = 2; `budgetedContext` = false.

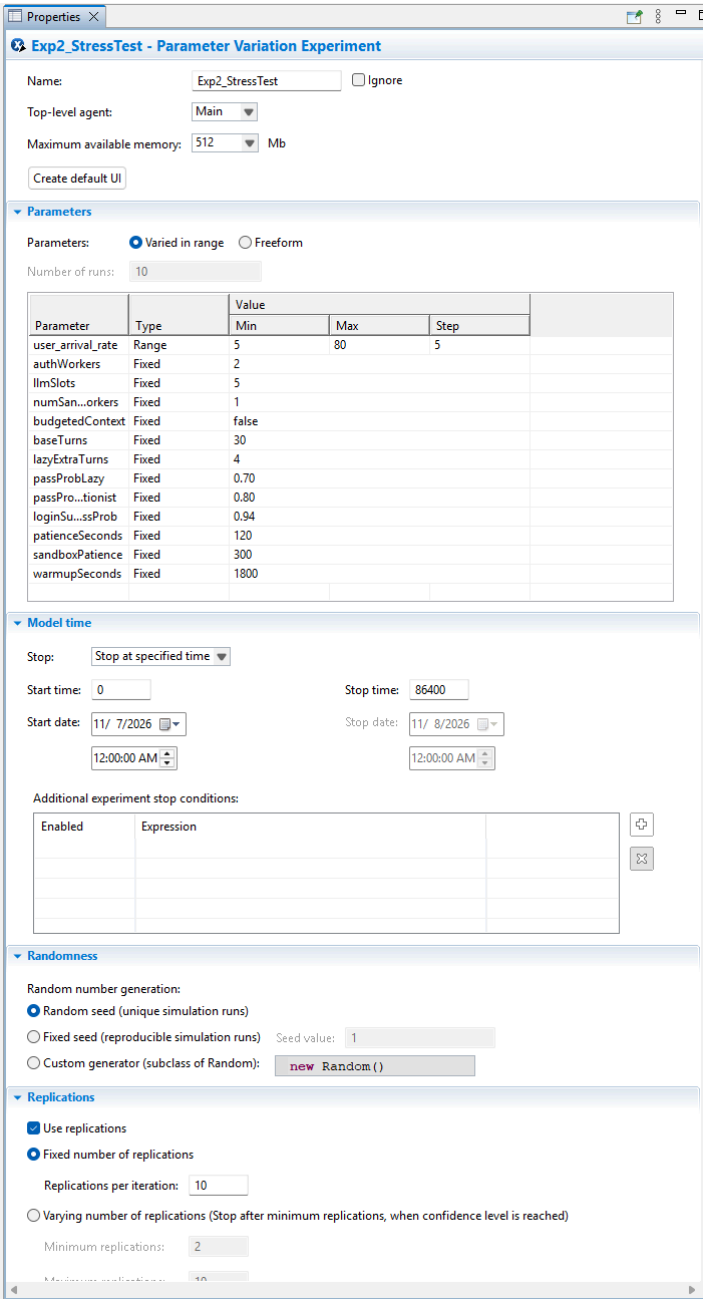
Figure 7. Dashboard at the end of Experiment 1.

Analysis: Under the baseline load the platform runs smoothly. Averaged over the 10 runs, the LLM pool is busy 22.2% of the time and the sandbox 16.8%, so no resource comes close to the 90% alarm level. No LLM queue forms (average wait 0.03 s; the longest queue seen in any run was 4 students), turn responses take 13.0 s, essentially the 13 s mean LLM call, and no student abandoned in any of the 10 runs (0.00%). The only waiting is at the sandbox, 6.2 s on average, because students whose tests fail return for a second or third grading. An average session lasts 37.3 minutes and throughput is 9.99 sessions/h, equal to the arrival rate. The run is the reference for the other experiments.

Experiment 2: The Stress Test (“Launch Day”)

Question: What happens if user traffic suddenly spikes?

Setup: Parameters Variation experiment with the capacities of Experiment 1; `user_arrival_rate` varied from 5 to 80 students/h in steps of 5; 24 h per run; 10 replications per value (160 runs).


Properties
Exp2_StressTest - Parameter Variation Experiment
Name: Exp2_StressTest
Ignore
Top-level agent: Main
Maximum available memory: 512 Mb
Create default UI
Parameters
Parameters: Varied in range Freeform
Number of runs: 10
Parameter Type Value Min Max Step
user_arrival_rate Range 5 80 5
authWorkers Fixed 2
llmSlots Fixed 5
numSan...orkers Fixed 1
budgetedContext Fixed false
baseTurns Fixed 30
lazyExtraTurns Fixed 4
passProbLazy Fixed 0.70
passPro...tionist Fixed 0.80
loginSu...ssProb Fixed 0.94
patienceSeconds Fixed 120
sandboxPatience Fixed 300
warmupSeconds Fixed 1800
Model time
Stop: Stop at specified time
Start time: 0
Stop time: 86400
Start date: 11/ 7/2026
Stop date: 11/ 8/2026
12:00:00 AM
12:00:00 AM
Additional experiment stop conditions:
Enabled Expression
Randomness
Random number generation:
Random seed (unique simulation runs)
Fixed seed (reproducible simulation runs)
Seed value: 1
Custom generator (subclass of Random):
new Random()
Replications
Use replications
Fixed number of replications
Replications per iteration: 10
Varying number of replications (Stop after minimum replications, when confidence level is reached)
Minimum replications: 2

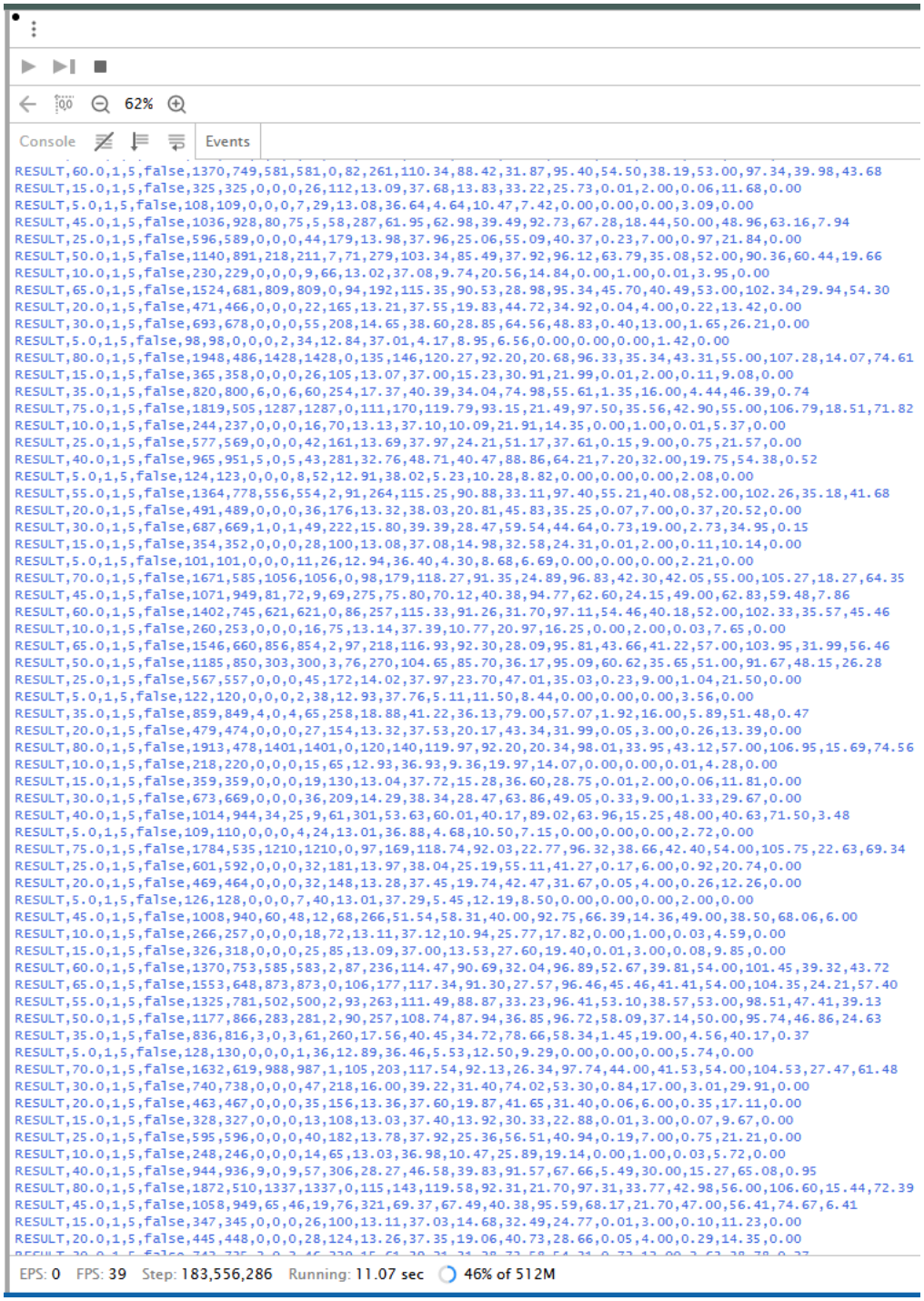


Figure 8. Experiment 2 settings and output.

Analysis: Abandoned_Users stays below 1 per run up to 25 students/h, is still only 1.65% of students at 40/h, and then rises steeply: 7.7% at 45/h, 47.3% at 60/h and 71.6% at 80/h (Graph 1). The system therefore breaks at **45 students/h**, the first rate above the 5% limit. The primary bottleneck is the LLM pool, which reaches 93.6%

utilization at 45/h and about 96–97% beyond, while the sandbox never exceeds 64.6% (Graph 3). Details are in Section 5.2.

Experiment 3: Optimization & Scaling

Question: How can the bottleneck from Experiment 2 be fixed so that the stress-test load is handled?

Setup: `user_arrival_rate` fixed at the broken level of **60 students/h** (47.3% abandonment in Experiment 2). An Optimization experiment minimized `llmSlots + numSandboxWorkers` (total resource units) with `llmSlots` from 5 to 12 and `numSandboxWorkers` from 1 to 4, subject to abandonment ≤ 5% and average session time ≤ 60 minutes. All 32 combinations were evaluated with 5 replications each.

- **Option A (scale resources):** `budgetedContext` = false; the optimizer finds the minimum LLM slots and sandbox workers.
- **Option B (optimize the application):** `budgetedContext` = true, so E1 shortens each LLM call from a mean of 13 s to about 10.3 s; the optimizer again finds the minimum resources.

Analysis: Option A needs 7 LLM slots and 2 sandbox workers (9 units); Option B needs 6 and 2 (8 units). Both reach the targets from the 6-unit baseline; Option B is one unit cheaper. See Section 5.2.

Note on the data: the first Option B optimization run stopped with an internal OptQuest error (demo engine) and was repeated; only the complete second run is reported.

4.3 Performance Metrics (Outputs)

Collected after a 30-minute warm-up:

Code	Metric	Definition
M1	Average response time (turn)	Time from the student's reply to the team's answer, including waiting (s).
M2	Average response time (session)	Time from login to exit for completed students (min).
M3	System throughput	Completed sessions per hour.
M4	LLM pool utilization	Share of time the LLM slots are busy (%).
M5	Sandbox pool utilization	Share of time the sandbox workers are busy (%).
M6 / M7	Queue length	Average and maximum number of turns waiting for an LLM slot.
M8 / M9	Waiting time	Average wait for an LLM slot and for a sandbox worker (s).
M10	Abandoned_Users	Students who left after waiting too long for an LLM slot or a sandbox worker (the failure count).
M11	Abandonment rate	Abandoned_Users divided by all students who finished or abandoned, in percent (the failure rate).

Acceptance thresholds used in the analysis: abandonment ≤ 5%; resource utilization ≤ 90%; average session time ≤ 60 min.

4.4 Simulation Run Parameters

- **Warm-up period:** the first 30 minutes (1,800 s) of every run are excluded; all counters and statistics are reset at that moment, so every reported value covers the same interval.
- **Run length:** every run covers 24 virtual hours (86,400 s).
- **Replications:** Experiments 1 and 2 use 10 replications per configuration; Experiment 3 uses 5 per combination of the optimizer. All use different random seeds, and the mean is reported.
- **Sampling:** utilization and queue lengths are sampled every virtual minute.

5. Results and Analysis

5.1 Data Presentation

All values are means over the replications of each configuration. Codes M1–M11 are defined in Section 4.3. Source: the result files of Experiments 1–3.

Table 1. Summary of scenarios and key metrics (Experiment 3 columns are at 60 students/h).

Metric	Exp 1 baseline (10/h)	Exp 2 at 30/h	Exp 2 at 45/h	Exp 2 at 60/h (broken)	Exp 3 Option A (7 slots + 2 sbx)	Exp 3 Option B (6 slots + 2 sbx)
M1 Turn response (s)	13.0	15.3	68.9	113.8	42.1	28.8
M2 Session time (min)	37.3	38.9	67.2	89.9	52.5	45.6
M3 Throughput (/h)	9.99	30.2	40.3	30.9	58.2	59.7
M4 LLM utilization (%)	22.2	67.0	93.6	96.2	92.1	87.6
M5 Sandbox utilization (%)	16.8	49.2	64.6	51.1	48.1	49.0
M7 Max LLM queue (students)	1.5	12.2	48.6	53.9	51.4	46.4
M10 Abandoned_Users (per run)	0	1.5	79.7	654.2	0	2.2
M11 Abandonment (%)	0.00	0.21	7.74	47.30	0.00	0.15

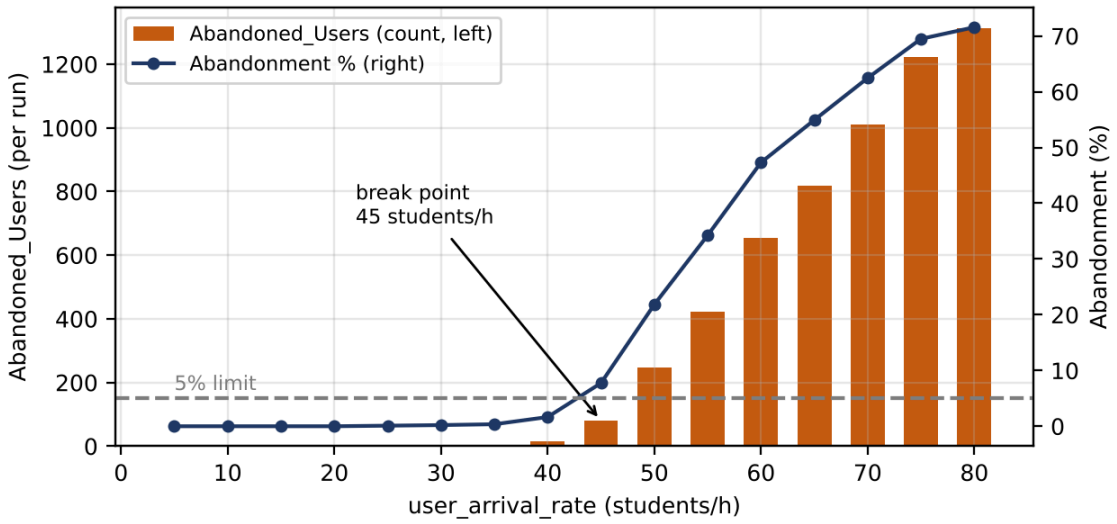
Table 2. Experiment 2: stress test, averages of 10 replications per arrival rate.

user_arrival_rate (/h)	M10 Abandoned_Users	M11 Abandon. (%)	M1 Turn response (s)	M4 LLM util. (%)	M5 Sandbox util. (%)	M3 Throughput (/h)
5	0.0	0.00	13.0	10.6	8.1	4.8
10	0.0	0.00	13.0	23.1	17.3	10.5

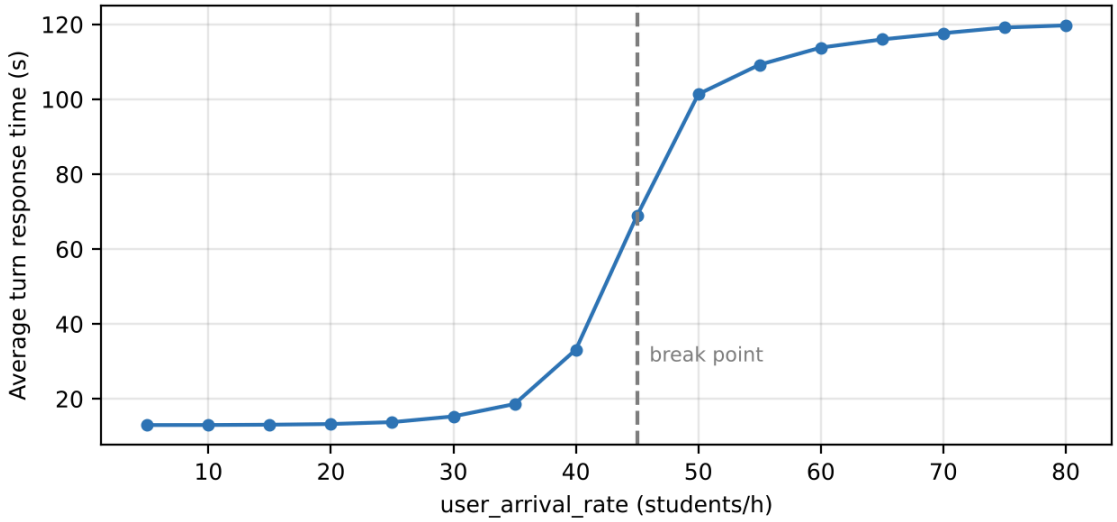
user_arrival_rate (/h)	M10 Abandoned_Users	M11 Abandon. (%)	M1 Turn response (s)	M4 LLM util. (%)	M5 Sandbox util. (%)	M3 Throughput (/h)
15	0.0	0.00	13.1	32.0	23.9	14.7
20	0.0	0.00	13.3	42.4	32.0	19.5
25	0.6	0.10	13.8	54.3	40.8	24.7
30	1.5	0.21	15.3	67.0	49.2	30.2
35	3.0	0.36	18.6	76.9	57.4	35.0
40	15.4	1.65	33.1	86.5	63.5	38.9
45	79.7	7.74	68.9	93.6	64.6	40.3
50	246.9	21.83	101.4	95.3	61.0	37.5
55	423.6	34.24	109.2	95.8	56.2	34.5
60	654.2	47.30	113.8	96.2	51.1	30.9
65	818.7	54.93	116.0	96.3	48.5	28.5
70	1011.6	62.46	117.7	96.9	42.4	25.8
75	1221.8	69.48	119.2	97.0	36.8	22.8
80	1313.1	71.57	119.7	97.2	36.6	22.2

Shaded rows: abandonment above the 5% limit (the system is broken).

Graph 1. Abandoned_Users versus arrival rate (10 replications per point)



Graph 2. Average turn response time versus arrival rate



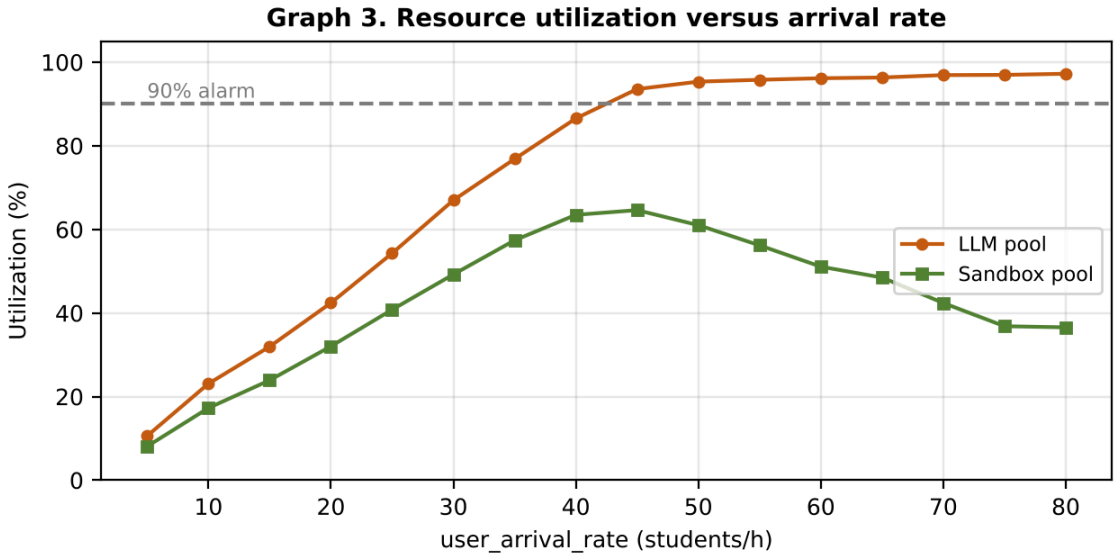
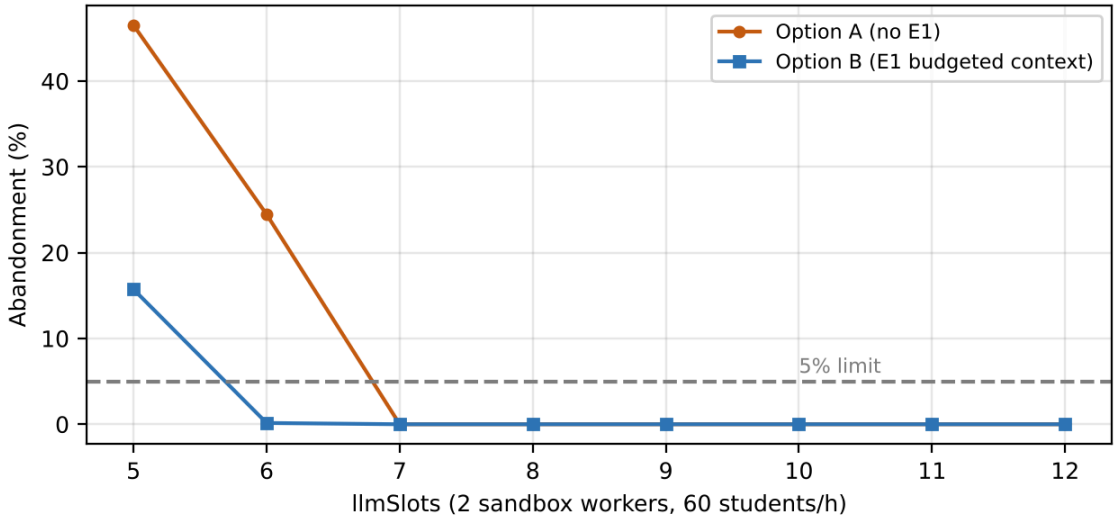


Table 3. Experiment 3: optimization results at 60 students/h (mean of 5 replications per configuration).

Configuration	`llmSlots`	`numSandboxWorkers`	Total units	M11 Abandon. (%)	M2 Session (min)	M3 Through-put (/h)	M4 LLM util. (%)
Baseline at the broken load (no E1)	5	1	6	47.66	89.3	30.7	96.3
Option A optimum (no E1)	7	2	9	0.00	52.5	58.2	92.1
Option A with safety margin	8	2	10	0.00	39.6	59.0	81.3
Option B optimum (E1 budgeted context)	6	2	8	0.15	45.6	59.7	87.6
Option B with safety margin	7	2	9	0.00	37.0	60.0	75.5

Graph 4. Abandonment versus LLM slots: Option A versus Option B



5.2 Analysis of Results

Experiment 1 (baseline). At 10 students per hour about six students are in the system at once and the platform runs smoothly: no LLM queue forms (average wait 0.03 s), turns take 13.0 s, abandonment is 0.00%, and no resource nears the 90% alarm level (LLM 22.2%, sandbox 16.8%). Students wait 6.2 s on average at the sandbox, the only place where a queue forms, because about a quarter of them need a second grading.

Experiment 2 (stress test). The utilization law predicts that the five LLM slots saturate at $\lambda^* = c \cdot 3{,}600 / (N \cdot E[S]) = 5 \times 3{,}600 / (32 \times 13) \approx 43$ students/h, where $N = 32$ is the average number of turns. The simulation agrees: abandonment is only 0.36% at 35/h and 1.65% at 40/h, crosses the 5% limit at **45/h (7.74%)**, and reaches 47.3% at 60/h and 71.6% at 80/h (Graph 1). Utilization shows the cause (Graph 3): the LLM pool rises from 22.2% at 10/h to 86.5% at 40/h and 93.6% at 45/h, and stays at 96–97% beyond, while the sandbox peaks at only 64.6%. The **LLM provider call limit is the primary bottleneck.** Once it saturates the LLM queue grows to an average of about 40 students, waits approach the 120 s patience limit (101 s average at 60/h), and students give up: of the 79.7 students who abandon per run at 45/h, 66.9 leave the LLM queue and 12.8 the sandbox queue, and at 60/h almost all (653 of 654) leave the LLM queue. Turn response climbs from 13.0 s to 33.1 s at 40/h, 68.9 s at 45/h and about 114–120 s at 60–80/h (Graph 2). Throughput peaks at 40.3 sessions/h at 45/h, then falls to 30.9 at 60/h and 22.2 at 80/h, because abandoned students never finish: past the break point extra arrivals make the platform serve fewer students, not more.

The sandbox is the second constraint. With a mean of 1.3 gradings of 50 s per student, one worker saturates at $3{,}600 / (1.3 \times 50) \approx 55$ students/h. This is confirmed in Experiment 3: at 60/h with one sandbox worker, abandonment stays at 13–15%

however many LLM slots are added (85–86% sandbox utilization, about 160 s average sandbox wait), and all of those students leave the sandbox queue. **Engines E1 and E2 are not constraints:** at 60 students/h with 32 turns each and 0.06 s per E1 call the engine is busy only 60/3,600 × 32 × 0.06 ≈ 3% of one worker, and authentication is below 1%.

Experiment 3 (optimization). At the broken load of 60 students/h the baseline of 5 LLM slots and 1 sandbox worker (6 units) loses 47.7% of students (session time 89.3 min). **Option A** needs 7 LLM slots and 2 sandbox workers (9 units, +3): abandonment falls to 0.00%, session time to 52.5 min and throughput to 58.2/h. **Option B** (E1 budgeted context) needs 6 slots and 2 workers (8 units, +2): abandonment 0.15%, session time 45.6 min, throughput 59.7/h. At the same capacity of 6 LLM slots and 2 workers, Option A still loses 24.5% of students while Option B loses 0.15% (Graph 4), because each E1-shortened call (mean 10.3 s instead of 13 s) occupies a slot for less time. Neither fix works with a single sandbox worker (13–15% abandonment even with 12 LLM slots), so the sandbox must grow with the LLM pool.

Safety margin. The strict optima are tight: the Option A optimum runs the LLM pool at 92.1%, above the 90% alarm level, with turn responses of 42 s, and the Option B optimum at 87.6% with 29 s. One extra LLM slot gives comfortable operation: Option A with 8 slots (10 units) reaches 81.3% utilization and 17.5 s turn responses; Option B with 7 slots (9 units) reaches 75.5% and 12.4 s. The one-unit advantage of Option B is the same with or without margin.

Cost judgment. Against the baseline of 6 units, Option A adds 3 units (+50%) and Option B adds 2 (+33%); with a safety margin the figures are +4 and +3. LLM slots and sandbox workers are assumed to cost the same per unit; if real prices differ, multiply each added unit by its price. Option B also costs engineering effort once (E1 is already designed and measured at about 0.06 s per call), whereas each added LLM slot is a recurring provider cost. Its trade-off is less transcript context per call, so persona fidelity should be re-audited at the smaller token budget. Option B (optimize the application) is therefore the cheaper choice.

Sensitivity (three calls per turn). If each of the three AI agents made its own LLM call for every student turn, the LLM load would triple and the saturation point would fall to about 43 / 3 ≈ 14 students/h. At the pilot size (8–10 students) the LLM pool would then run at about 70% utilization, so the pilot fits, but a class-size launch would not.

5.3 Model Validation

The model was checked against calculations that do not need the simulation, using the recorded results:

1. **Sanity check (no queueing).** At the lowest tested rate, 5 students/h, the session time must equal the sum of the mean service times: 32 turns × (51.7 s think + 13

s LLM) + 1.3 gradings × 50 s + 0.3 revisions × 280 s + small terms ≈ 2,220 s ≈ 37 minutes. The model gave 37.1 minutes (37.3 at 10/h), and the turn response was 13.0 s against the 13 s mean LLM call.

2. **Utilization law.** Expected LLM utilization = `user_arrival_rate` × 32 × 13 s ÷ (3,600 × `llmSlots`) = 23.1% at 10 students/h and 92.4% at 40/h. The model gave 22.2% (Experiment 1) and 23.1% (Experiment 2) at 10/h, and 86.5% at 40/h (a little lower because queueing and the first abandonments begin). Expected sandbox utilization at 10/h = 10 × 1.3 × 50 / 3,600 ≈ 18%; the model gave 16.8% and 17.3%.
3. **Input probabilities.** In Experiment 1 there were on average 15.0 login retries per 235 sessions (6.4%, input 6%) and 76.6 resubmissions per run (0.33 per session; expected 0.315 from the pass probabilities 0.70 and 0.80 with a limit of 3 attempts).
4. **Conservation and Little's law.** At 10/h throughput was 9.99 sessions/h against 10 arrivals/h, and Little's law gives about 10 × 37.3 / 60 ≈ 6 students in the system. In Experiment 1 arrivals minus completed minus abandoned students stayed within ±7; at high load it grows (up to about 60 students) because sessions of 90 minutes are still in progress when the run ends.
5. **Saturation point.** The analytic LLM saturation point is about 43 students/h. The observed break point (abandonment above 5%) is 45/h, where LLM utilization first exceeds 90% (93.6%) and throughput peaks (40.3/h) before falling.
6. **Extreme-value tests.** With 12 LLM slots and 4 sandbox workers at 60/h (Experiment 3, Option A) abandonment is 0.00% and the turn response is 13.1 s, as expected when capacity is ample; with 5 slots and 1 worker it is 47.7%.

Agreement between these independent calculations and the simulation output supports the logic of the model. Its absolute numbers remain only as reliable as the assumed service times (especially the LLM reply time, think time and sandbox time), which should be replaced by pilot measurements.

6. Conclusion

6.1 Summary of Findings

The simulation shows that Fix it with Scrum comfortably serves a pilot-sized load but breaks when too many students start in the same hour. At 10 students/h nothing is stressed (no abandonment, LLM pool 22%, sandbox 17%, 37-minute sessions). With the baseline capacity (5 LLM slots, 1 sandbox worker) abandonment exceeds the 5% limit at **45 students/h** (7.7%), close to the analytic estimate of 43, and reaches 47.3% at 60/h. The bottleneck is the number of concurrent LLM calls (utilization 94–97%); the

sandbox is the second constraint (about 55 students/h with one worker), and Engines E1 and E2 and authentication are not constraints.

At the broken load of 60 students/h both fixes restore service. Scaling resources (Option A) needs 9 units (7 LLM slots + 2 sandbox workers); using E1 budgeted context (Option B) needs 8 units (6 + 2), one unit fewer, and with a safety margin 9 against 10 units, so Option B is the cheaper fix. Limitations: LLM latency, think time, the provider limit and sandbox times are planning estimates; the model uses one LLM call per student turn; abandoning students never retry; and the optimizer used 5 replications per combination, so differences of one unit are small compared with the run-to-run variation. All should be recalibrated with pilot logs.

6.2 Recommendations

1. **Adopt E1 budgeted context in production.** Cap each agent turn's prompt at the 4,000-token budget. It shortens every LLM call (13 s to about 10.3 s), lowers LLM utilization and saves one resource unit compared with scaling alone.
2. **Size the LLM concurrency allowance for the peak hour.** Five concurrent calls support about 43 students per hour at most. For a launch-day load of 60 students/h, plan 6–7 concurrent calls with E1 (7–8 without it), including a margin to stay below 90% utilization, or stagger session starts.
3. **Run at least two sandbox workers for class-scale use.** One worker saturates near 55 students per hour and keeps 13–15% of students from finishing even when the LLM is ample.
4. **Protect students under overload.** Add admission control (a short waiting room for new logins) so that sessions in progress are served first, show an "agents are thinking" message, and let a student retry instead of ending the session after a timeout.
5. **Do not spend effort optimizing E1 and E2 run times.** They take milliseconds against seconds for the LLM; keep the greedy fallback in E1 only to protect worst-case inputs.
6. **Monitor and recalibrate.** Track LLM and sandbox queue lengths in production, replace the assumed service times with pilot measurements, test the three-calls-per-turn case, and rerun the three experiments before any wider release.

References

Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete-event system simulation* (5th ed.). Pearson.

Borshchev, A. (2013). *The big book of simulation modeling: Multimethod modeling with AnyLogic 6*. AnyLogic North America.

Law, A. M. (2015). *Simulation modeling and analysis* (5th ed.). McGraw-Hill Education.

Little, J. D. C. (1961). A proof for the queuing formula: L = λW. *Operations Research, 9*(3), 383–387.

Qureshi, K., Martin, G., & Peng, Y. (2026). Budget-aware routing for long clinical text. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 42583–42598). Association for Computational Linguistics.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org

The AnyLogic Company. (n.d.). *AnyLogic 8 in three days* and *AnyLogic Process Modeling Library help*. https://www.anylogic.com